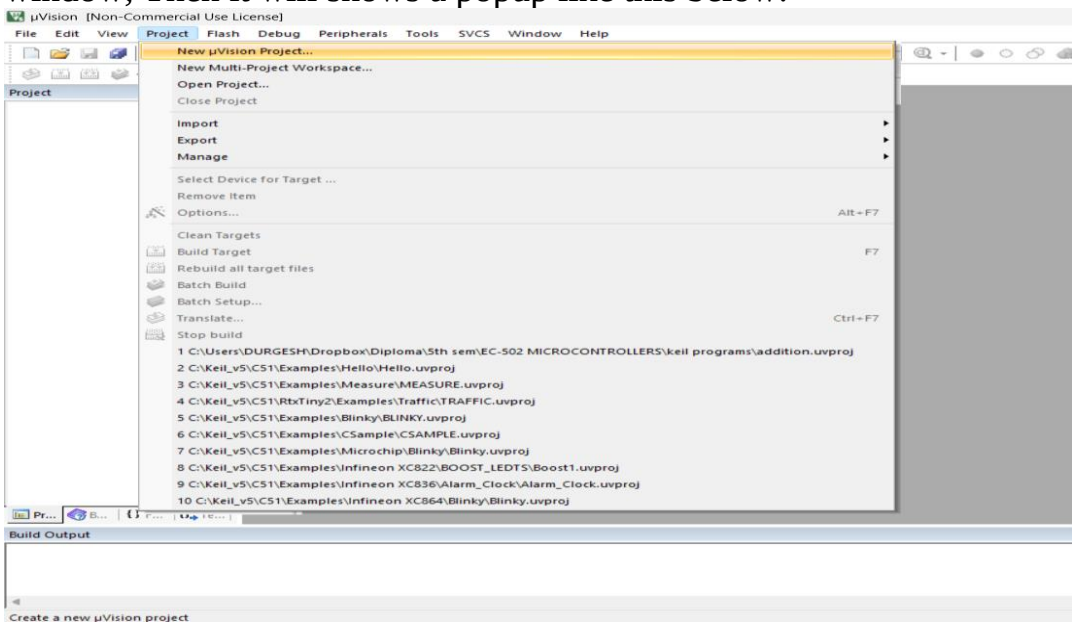
Microcontroller Practical / Theory

Keil uVision 5 IDE

Steps to use Keil uVision Software

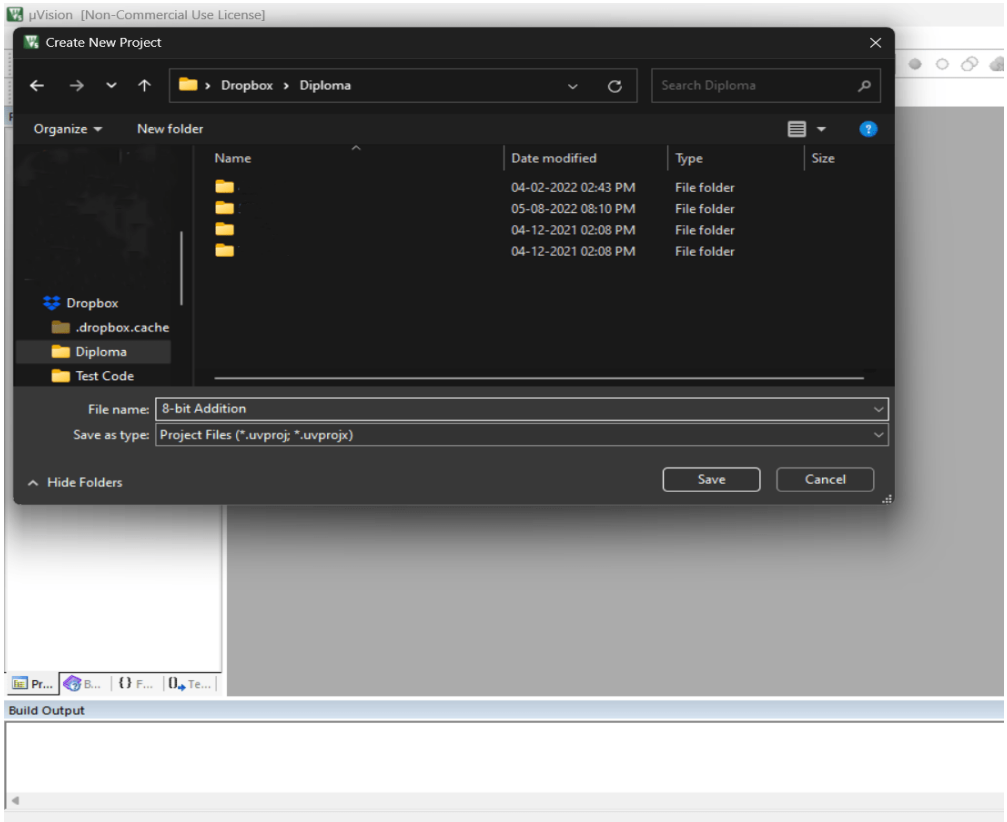
After Installing the Keil software follow the below steps to perform the 8-bit Addition

1. Open the software and click on the Project option which is settled at the top of the window, Then it will shows a popup like this below:



Keil uVision Software

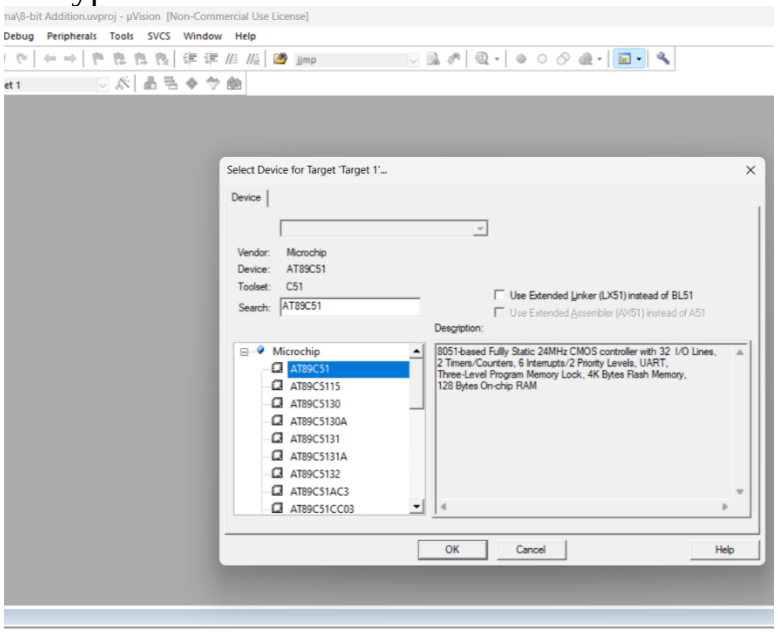
And then click on the **"New uVision Project"** option then it will opens and explorer to save it like below:



Saving the Project

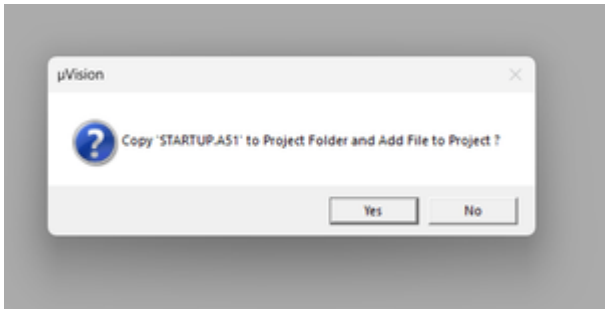
Give a name to your Project and save it in your desired location. Then a popup will come to select the device of Microcontroller here in this case we are going to select

the '**AT89C51**' Microcontroller to run the program. In the popup there will be a search bar type the above name then select it as below:



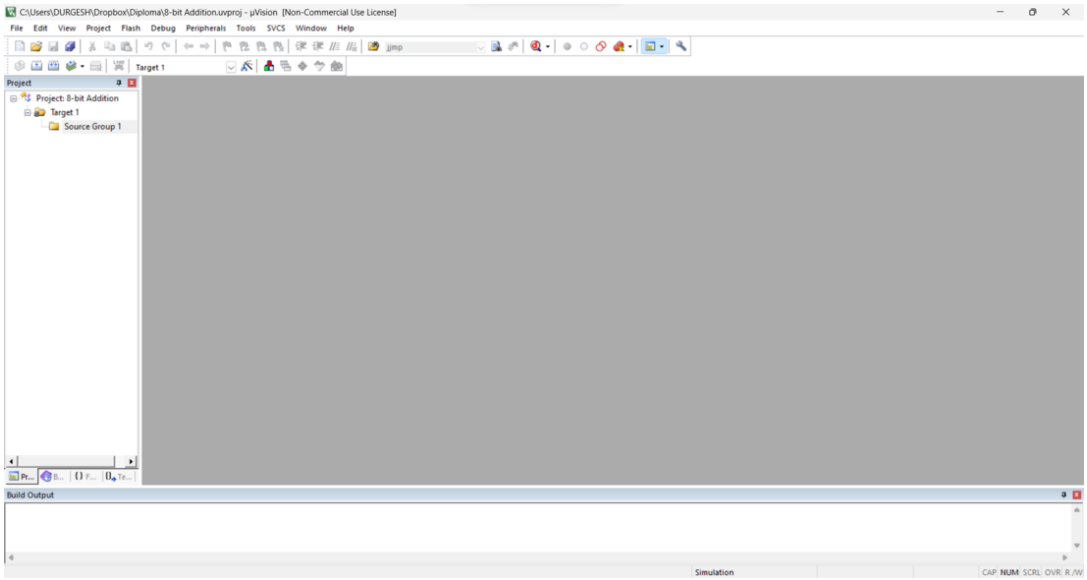
Select the MC AT89C51

After selecting the MC click on "**OK**" Option 'AT89C51' is a Microcontroller that designed by 'ATML' it is 8051-MC. Then you will get a dialog box with yes or no like below:

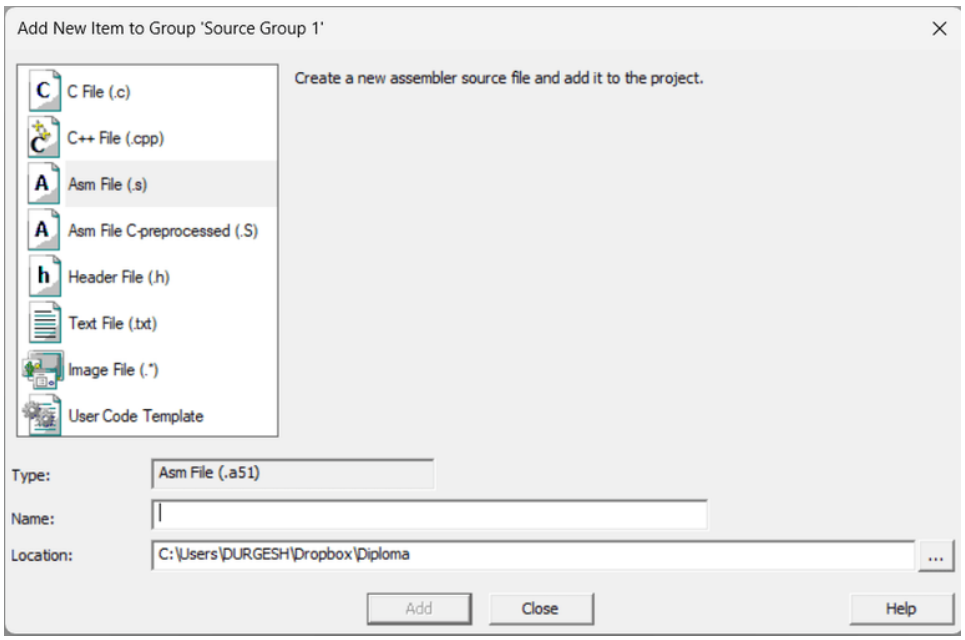


Click on NO

Click on '**NO**' we don't have need of that file so that click on NO but that's up to you if you have any use cases with that file you can click on YES Then we have successfully created a project file like below:



Then right click on **Source Group 1** and click on the option called **"add new item to Group 'Source Group 1'"** and select the file type and enter the file name also like below:

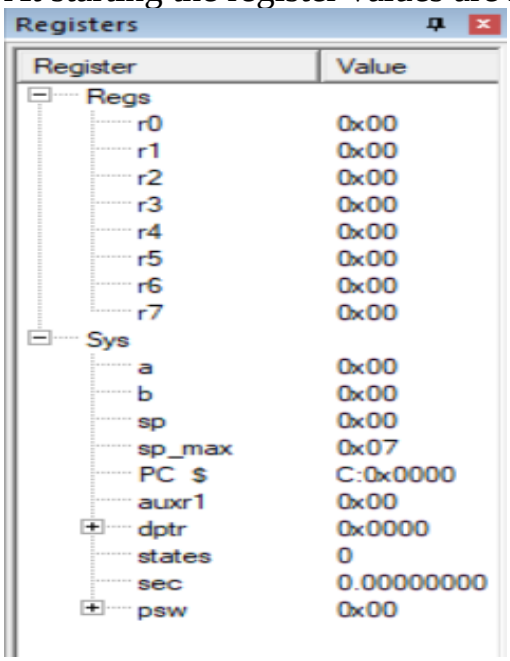


Then click on Add that create an .asm file in your project. Now the asm file will be open in the editor to type the program so that Enter the above Program of 8-bit Addition. You can paste the above Program in your code editor after completing the code click on **"F7" key** to Build the target file or right click on the asm file and select Build target option from there.

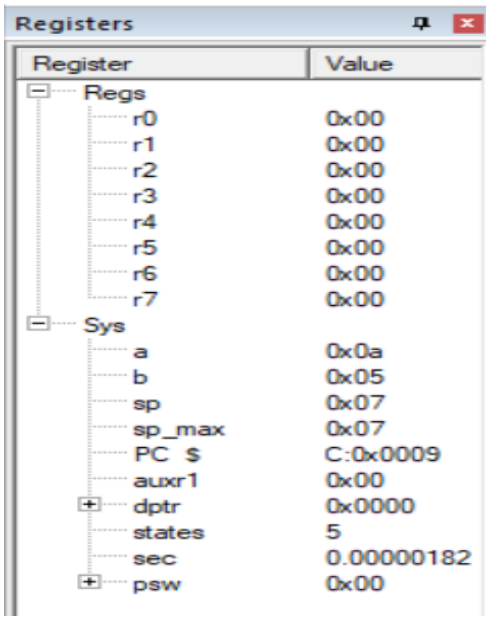
In the below at Build output box you will get like below:

```
Build started: Project: addition
Build target 'Target 1'
assembling add.asm...
linking...
Program Size: data=8.0 xdata=0 code=11
".\Objects\addition" - 0 Error(s), 0 Warning(s).
Build Time Elapsed:  00:00:01
```

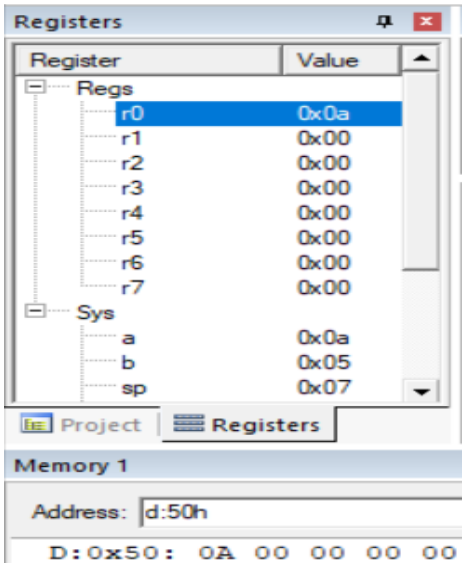
So when we are getting the output as **0 Errors** we can proceed to run the Program by debugging.
To debug the program we can simply click **'ctrl + F5' or** there will be a option to do that at the top of the window.
we can debug the program by step wise or complete program at a time by clicking on keys, By clicking on **F11 Key** we can achieve step wise debugging
At starting the register values are like below:



At starting all the registers are empty by pressing the **F11 key** we can check how the values of the accumulator, b are changing.



We can in the above picture that the value #5 came into Accumulator and B also added, the result was stored in Accumulator as per the program, as in the program we are added that to store the value in **r0** register and **50h** address see the result in the below:



As you can see the result was stored in R0 Register and 50h address. so that we can change the program to test your own values and make sure to rebuilt it again by clicking on **F7 key** .

Embedded C programming

What is Embedded C programming and how is it different?

C is a high-level programming language intended for system programming. **Embedded C** is an extension that provides support for developing efficient programs for **embedded devices.** Yet, it is not a part of the **C language.** In this "Embedded C programming" article, we shall discuss the following topics.

Table of Content

What is Embedded C Programming

Embedded C programming language is an extension to the traditional **C programming language**, that is used in embedded systems. The embedded C programming language uses the same syntax and semantics as the C programming language.

The only extension in the Embedded C language from normal C Programming Language is the I/O Hardware Addressing, fixed-point arithmetic operations, accessing address spaces, etc.

Now will move on to the Difference between C and Embedded C.

Difference between C and Embedded C:

C Programming Language	Embedded C Programming Language
In nature, it is a native development	In nature, It is cross-development
It is independent of hardware architecture	It is hardware dependent
Used for desktop application	Used for limited resources like RAM and ROM

Now let's learn about the basic structure of Embedded C program.

Basic Structure of Embedded C Program:

The embedded C program has a **structure similar to C programming.**

The five layers are:

1. Comments
2. Pre-processor directives
3. Global declaration
4. Local declaration
5. Main function()

The whole code follows this outline. Each code has a similar outline. Now let us learn about each of this layer in detail.

```
1   Multiline Comments . . . . . Denoted using /*……*/
2   Single Line Comments . . . . . Denoted using //
3   Preprocessor Directives . . . . . #include<…> or #define
4   Global Variables . . . . . Accessible anywhere in the program
5   Function Declarations . . . . . Declaring Function
6   Main Function . . . . . Main Function, execution begins here
7   {
8   Local Variables . . . . . Variables confined to main function
9   Function Calls . . . . . Calling other Functions
10  Infinite Loop . . . . . Like while(1) or for(;;)
11  Statements . . . . .
12  ….
13  ….
14  }
15  Function Definitions . . . . . Defining the Functions
16  {
17  Local Variables . . . . . Local Variables confined to this Function
18  Statements . . . . .
19  ….
20  ….
21  }
```

Comment Section:

Comments are simple readable text, written in code to make it more understandable to the reader. Usually comments are written in // or /* */.

Example: //Test program

Let's look into Preprocessor Directives Section.

Preprocessor Directives Section:

The Pre-Processor directives tell the compiler which files to look in to find the symbols that are not present in the program.

For Example, in 8051 Keil compiler we use,

```
1    #include<reg51.h>
```

Let's look into Global declaration Section.

Global Declaration Section:

This part of the code is the part where the global variables are declared. Also, the user-defined functions are declared in this part of the code. They can be accessed from anywhere.

```
1    void delay (int);
```

Let's look into Local declaration section.

Local Declaration Section:

These variables are declared in the respective functions and cannot be used outside the main function.

Let's look into the Main function section.

Main Function Section:

Every C programs need to have the main function. So does an embedded C program. Each main function contains 2 parts. A declaration part and an Execution part. The declaration part is the part where all the variables are declared. The execution part begins with the curly brackets and ends with the curly close bracket. Both the declaration and execution part are inside the curly braces.

Function Definition Section

In this section, the function is defined.

```
1   void main(void) // Main Function
2   {
3   P1 = 0x00;
4   while(1)
5   {
6   P1 = 0xFF;
7   delay(1000);
8   P1 = 0x00;
9   delay(1000);
10  }
11  }
```

This is the basic structure of the embedded c program.

With this, we come to an end of this “Embedded C Programming” article. I hope you have understood the basic structure.

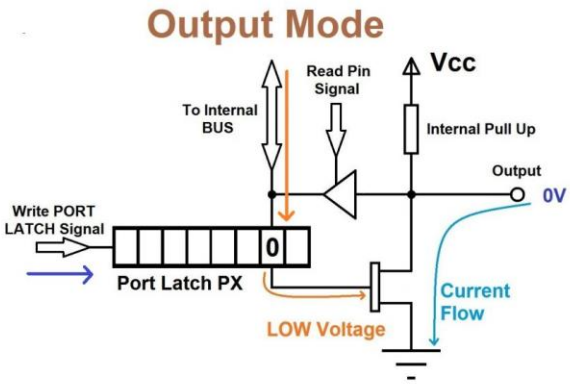
LED Blinking using 8051 Microcontroller and Keil C – AT89C51

By Ebin George 8051 Microcontroller, Electronics, Tutorials 8051 Microcontroller, Embedded, Microcontroller, Proteus 38 Comments

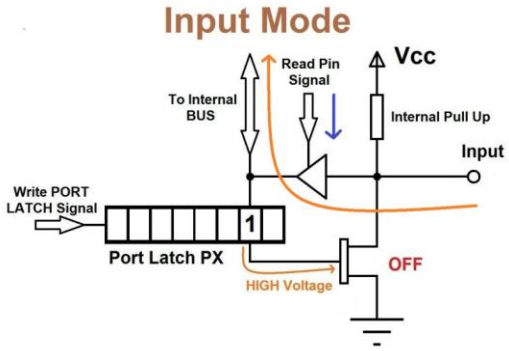
8051 Microcontroller is a programmable device which is used for controlling purpose. Basically 8051 controller is Mask programmable means it will programmed at the time of manufacturing and will not programmed again, there is a derivative of 8051 microcontroller, 89c51 micro controller which is re-programmable.

AT89C51

89c51 is 8-bit device means it is capable of doing 8-bit operations. It have 4 ports which are used as input or output according to your need. This device also have Timer, Serial Port interface and Interrupt controlling you can use these according to your need. The datasheet may be downloaded from here.



8051 Ports Explained Note : There is no Internal PULL UP resistors for Port P0



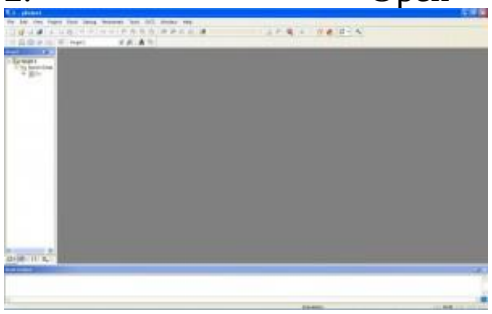
8051 Port in Output Mode Note : There is no Internal PULL UP resistors for Port P0

8051 Port in Input Mode

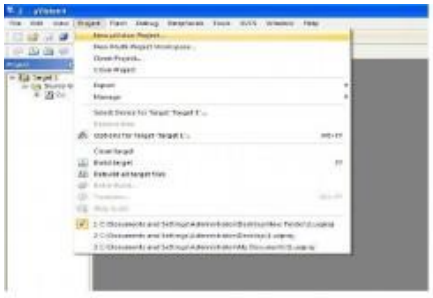
Using Keil uVision 4

1. Download and Install Keil uVision4

2. Open Keil uVision



3. Create a new Project : Project >> Create µVision Project



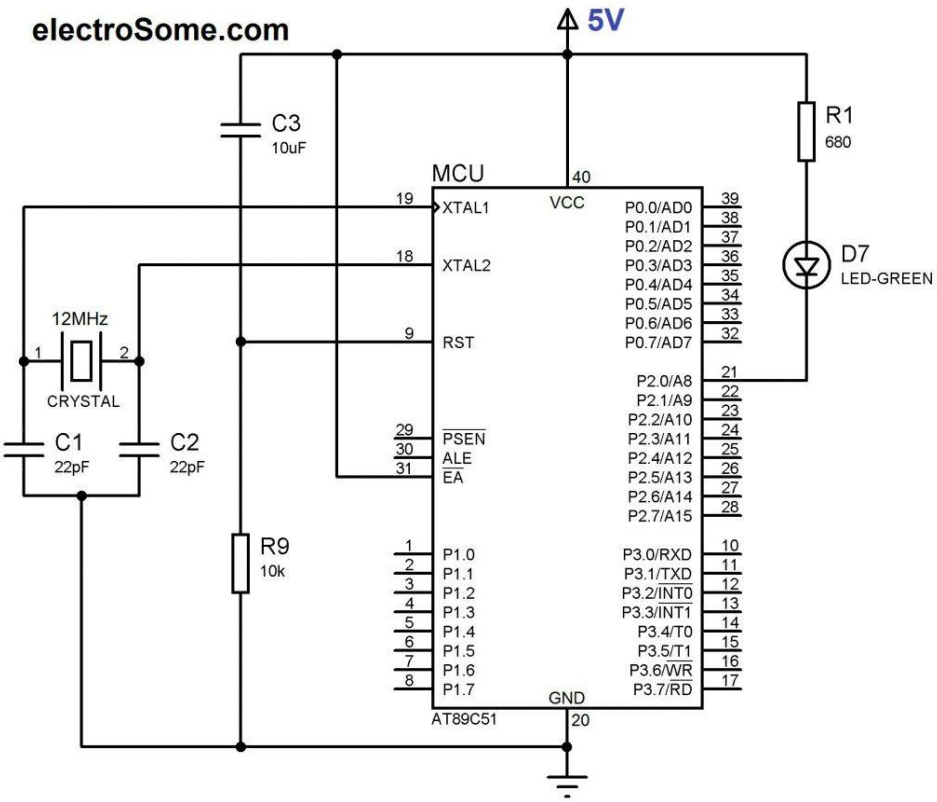
4. Browse for the location
5. Select the microcontroller Atmel>>AT89C51

6. Don’t Add The 8051 startup code
7. **File>>New**
8. Adding Hex file to the output

Right click on **Target1>>options for target “target 1”**
In the Output Tab check the “Create HEX file” box<

- To change the operating frequency goto **Target** tab on the window obtained by right clicking on **Target1>>options for target “target 1”**

Circuit Diagram



LED Blinking with 8051 Microcontroller – AT89C51

Keil C Program

```
#include<reg52.h>          // special function register declarations
                           // for the intended 8051 derivative

sbit LED = P2^0;           // Defining LED pin

void Delay(void);          // Function prototype declaration

void main (void)
{
   while(1)                // infinite loop
   {
      LED = 0;             // LED ON
      Delay();
      LED = 1;             // LED OFF
      Delay();
   }
}

void Delay(void)
{
   int j;
   int i;
   for(i=0;i<10;i++)
   {
      for(j=0;j<10000;j++)
      {
      }
   }
}
```

1. Enter the source code.

2. Save it

3. Then Compile it. Click Project>>Build Target or F7

The hex file will be generated in your Project Folder.

AT89C51 needs an oscillator for its clock generation, so we should connect external oscillator. Two 22pF capacitors are used to stabilize the operation of the Crystal Oscillator. EA should be strapped to VCC for internal program executions. AT89C51 has no internal Power On Reset, so we have to do it externally through the RST pin using Capacitor and Resistor. When the power is switched ON, voltage across capacitor

will be zero, thus voltage across resistor will be 5V and reset occurs. As the capacitor charges voltage across the resistor gradually reduces to zero.

This pin also receives the 12-volt programming enable voltage (VPP) during Flash programming, for parts that require 12-volt VPP.